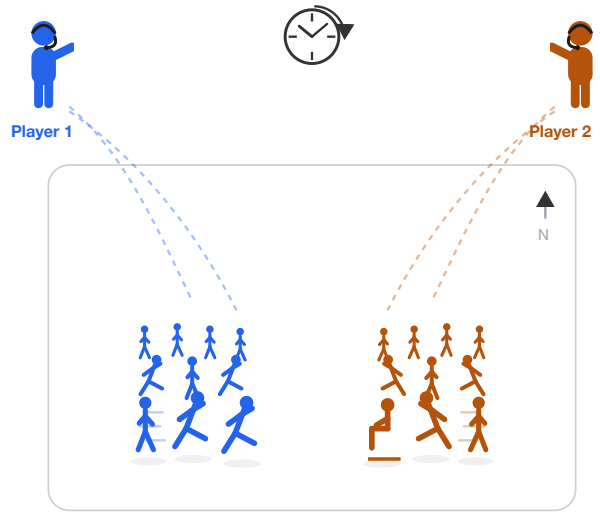
World Commander

Natural-language command of agent crowds in real-time strategy games, under latency and memory budgets

This proposal grew from Yubo Huang's interest and insight, under the guidance of Dr. Enmao Diao.



What we build: real-time, language-commanded agent crowds (here, two players commanding their own).

The long-term goal: a new kind of video game, played by voice from the commander's chair.

Image generated with GPT Image 2.

June 2026

Outline

1. **Related Work**: agents, real-time benchmarks, and efficient inference
2. **The Roadmap**: one question, across steadily harder environments
3. **The Command Arena**: the first step, before StarCraft II

1 Related Work

- The interface shipped once (Tom Clancy's EndWar (2008)), but on a 70-word grammar. Language models lifted that limit; they still **cannot act at game pace**.
- LLMs already play StarCraft II, by text (TextStarCraft II (NeurIPS 2024)) or screenshots (AVA (2026)), but only by **pausing or slowing the clock**.
- Real-time is only now being measured (VideoGameBench (2025): performance drops once the clock runs), and only for single agents, **not the efficiency methods**.
- Eviction, quantization, and distillation are tuned on perplexity, where they already drop the wrong instruction (Pitfalls of KV Cache Compression (ACL 2026)); none is **tested inside a running game**, where a discarded detail can decide the match.

> **How much does command cost, in latency and memory, at game speed, and how can it be reduced?**

Related Work: The Landscape (1 of 2)

Agents at game speed, and the methods under test. The fuller field behind the argument; the canonical review with every link is `LITERATURE.md`.

Agents in real-time games. AlphaStar (Nature 2019) mastered StarCraft II by full reinforcement learning (no language, datacentre scale, fully autonomous); the precedent we deliberately do not repeat. LLMs now issue SC2 commands (TextStarCraft II, LLM-PySC2, AVA), but only with the clock paused or slowed. Real-time play has only recently been benchmarked: VideoGameBench and AgileThinker show performance dropping under time pressure. Closest to us, HLA, Adaptive Command, and DPT-Agent put a human in command with a fast-slow split, but cooperatively, or by steering a hand-built behaviour tree. CivRealm is the turn-based-strategy contrast.

Efficient inference, under latency and memory. The methods evaluated here: KV-cache eviction (H2O, SnapKV, StreamingLLM, OBCache), plus action-level Speculative Actions, which cuts latency losslessly. Recent analyses (Pitfalls of KV Cache Compression, DefensiveKV, SideQuest) show eviction can discard the wrong information, and that its effect is task-dependent. WorldMemArena pushes agent-memory evaluation into a closed interaction loop: a different "memory" from the KV-cache, but the same methodological turn we make.

Related Work: The Landscape (2 of 2)

Fast-slow execution, game-state representation, and the motion stack.

Hierarchical and VLA fast-slow execution. π0 pairs a slow vision-language brain with a fast action expert (up to 50 Hz); Fast-in-Slow folds both into one backbone; OpenVLA is the open "tokens-as-actions" baseline. This is the split we adapt for commander plus executors. Language-conditioned precedents: NL-to-StarCraft II grounding (2019) and CALVIN.

Game-state representation and world models. Toward fewer tokens per decision: Diao's graph tokenization (ICLR 2026), VQ-VAE, and IRIS / Δ-IRIS (context-aware delta tokenization). Action-conditioned world models, WHAM / WHAMM (Nature 2025), GameNGen, and DreamerV3, are the engines behind any what-if forecaster.

Crowd and real-time motion (embodiment, deferred). CrowdMoGen has an LLM plan a crowd's motion, but makes no real-time claim. Per-character real-time generators under budget: MotionLCM, MotionPCM, MotionStreamer, and MotionBricks (NVIDIA, SIGGRAPH 2026; 350k skills from one backbone). Single-character today; language-commanded crowds on one GPU remain open.

2 The Roadmap

One program, one question: what does command cost at game speed, and how can it be reduced? That question holds across the whole program; the one thing that grows is the environment we test it in:

Toy Arena → StarCraft II → Multiplayer → Full Game

Throughout, one loop: **measure** the cost of command, **build methods** to cut it, behind a single **commander interface**. Three milestones fall out along the way, each a paper a community already wants:

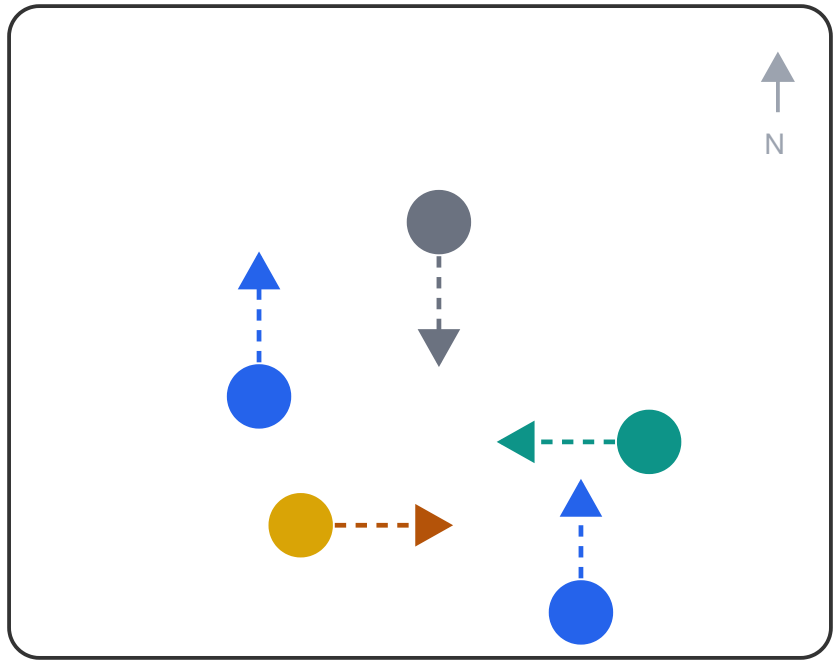
- **Real-time commander benchmark:** the cost, measured against a live clock (KV-cache and pruning)
- **Game-state tokenizer:** fewer tokens per decision, so less latency and memory (tokenization beyond text)
- **Crowd-motion embodiment:** command at scale on one GPU, deferred (motion generation and graphics)

The end-state game is the long-term goal, not a deliverable.

3 The Command Arena

The first step: a warm-up task that validates the streaming-command infrastructure cheaply, before StarCraft II.

- Colour-tagged agents in a room; each moves in **one of four directions** on command.
- One command is trivial; the test is the **stream**: many, fast.
- Harder with **rate**, **grouping** ("everyone except the yellow one"), and **memory** ("the one I sent west earlier, now move it north").
- **Not single-sided:** uncontrolled agents move on their own, so a late command concedes ground.
- Measured: **grounding accuracy**, **command-to-action latency**, **deadline misses**.



The command arena. Colour-tagged agents, each moving in one of four directions on command.

Team

Who builds what, across the command-to-action loop.

Member	Focus
Yubo Huang	Command interpreter, benchmark and harness, the budget framing
Enmao Diao (advisor)	Efficient inference: KV-cache eviction, pruning, tokenization
Hongsong Tang (Tencent Timi Studio)	Multi-agent coordination (RL / MARL)
Xirui Shi (University of Alberta, Amii)	Embodiment and VLA (vision-language-action) execution: model-generated motion for commanded units